Project Brief: Build Something Big

Your Level 3 final project, from idea to showcase

Name (nickname is fine): _____ Date: _____

Tip: What is this?

For the next four weeks you will design and build a program that is bigger than anything you have made so far — and you will build it like real software makers do: plan first, save your progress with git, test it with pytest, ask people to try it, and explain your design. You can work alone or in a pair.

1. Choose your option

Option	What you make	Start from
(a) Grid game	A game on a grid of squares: a maze, a dungeon, a puzzle, a farm. Characters are objects; enemies can use an algorithm to move.	`grid_game_template.py`
(b) Generative art app	An app that makes pictures from rules: fractals, patterns, tiles. Shapes are objects; your art can be saved and loaded again.	`art_app_template.py`
(c) Data / AI app	An app that answers questions about data, draws charts, or predicts things with a nearest-neighbor "AI".	`data_app_template.py` and `data/games.csv`

The templates are in the `starter` folder. They already run; the `README.md` there tells you how to copy the folder and make your first git commit.

2. The must-haves

Whatever you build, your finished project must have all of these. Check them off as you go.

Self-check: tick each box when it is true

- ☐ **At least 2 classes** that you wrote, and one of them uses **inheritance** (`class Dragon(Character):`) or **composition** (a class that has other objects).
- ☐ A **data structure beyond a flat list**: a dictionary, a stack or queue, or a 2D grid.
- ☐ One **named algorithm**: a search, a sort, recursion, or breadth-first search (BFS).
- ☐ A **saved file**: high scores, a level, saved art or results, created if it is missing.
- ☐ At least **5 pytest tests** that pass.
- ☐ A **git history with 8 or more meaningful commits** (small steps with clear messages).
- ☐ A short **README.md**: what it is, how to run it, how it is built, credits.
- ☐ **Fair and safe**: nicknames only, no personal data, never color alone (use shapes, letters or words too), credit for anything you did not make.

3. Idea menu

Pick one, mix two, or invent your own. Small and finished beats huge and broken!

(a) Grid games	(b) Generative art	(c) Data / AI
Dungeon crawler: collect keys, open doors, a monster that hunts you with BFS	Fractal forest: trees with different depth, angle and season palettes; save your favorite settings	Game-night stats: survey your family's favorite games, clean the data, chart it
Sokoban-style box pusher with levels in text files and an undo stack	Quilt maker: tiles of shape classes in a grid, mirror and rotate rules	Monster sorter: a nearest-neighbor AI guesses the type of a new monster from its features
Snake on a grid with a queue for the body	Flood-fill coloring book: click a region to fill it (recursion or BFS)	Weather or reading log: average, best week, a bar chart, a prediction

(a) Grid games	(b) Generative art	(c) Data / AI
Farm or garden simulator: plants are objects that grow each tick	Spirograph studio: pattern classes, sliders by keys, a gallery saved as JSON	Story maker: a Markov text generator trained on your own stories

4. Milestones

Milestone	When	Done when…
M1 Pitch	end of week 9	Your pitch page (section 5) is filled in and your teacher has said "go". Your project folder exists and has its first commit.
M2 Design + core classes	week 10	CRC cards and a class diagram are drawn. Your main classes exist in code, run without errors, and have the first 2 tests. At least 3 commits.
M3 Features + 5 tests + playtest	week 11, first half	The must-have features work, 5+ tests pass, and someone else has playtested it (feedback card filled in).
M4 README + polish	week 11, second half	You fixed the top bugs from the playtest, wrote the README, and have 8+ meaningful commits.
Showcase	week 12	You show your project and explain one design decision.

Tip: Commit like a pro

Commit every time something small works: `"Add Hero class"`, `"Stop hero walking through walls"`, `"Test that treasure adds 1 point"`. Messages like `"stuff"` or `"final"` do not count as meaningful.

5. Design pages

Page A: My pitch (M1)

Project name _____

Option (a, b or c) _____

In one sentence, it is... _____

It is for (who will enjoy it?) _____

Must-have feature 1 _____

Must-have feature 2 _____

Must-have feature 3 _____

Nice-to-have (if there is time) _____

My algorithm (and what it does) _____

My data structure (and what it holds) _____

The file it saves _____

Page B: CRC cards (M2)

Write one card for each class. **Responsibilities** are what it knows (attributes) and does (methods). **Collaborators** are the other classes it uses.

Worked example: A CRC card for a grid game

Class: Dragon **Responsibilities:** knows its position and speed; chooses the next square using BFS; can say if it caught the hero. **Collaborators:** Grid (to find open squares), Hero (where to go). **Is a** Character (inherits position and `move`).

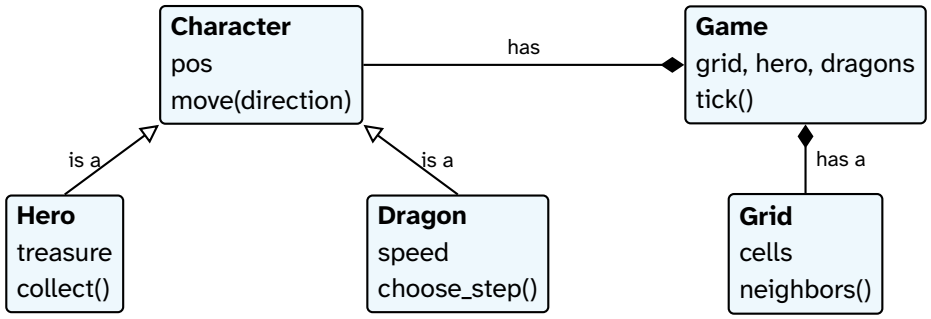
Class:	
Responsibilities (what it knows / does)	**Collaborators (works with)**

Class:	
Responsibilities (what it knows / does)	**Collaborators (works with)**

Class:	
Responsibilities (what it knows / does)	**Collaborators (works with)**

Page C: Class diagram

Draw a box for each class (name, attributes, methods). Draw an arrow with an **open triangle** for “is a” (inheritance) and a line with a **diamond** for “has a” (composition). Example:



Page D: My milestone plan

Milestone	Date	What will work (and which commits I expect)
M1 Pitch		
M2 Design		
M3 Features		
M4 Polish		
Showcase		

6. Testing your project

Test the **logic**, not the drawing. Keep rules (moving, scoring, saving, sorting, predicting) in classes and functions that do not use turtle, and keep the turtle code in its own part. Ideas:

- a wall blocks a move; collecting treasure adds a point;
- your search or sort gives the right answer on a tiny example *and* an edge case (empty list, unreachable square);
- saving and loading gives back the same data (use pytest's `tmp_path` folder);
- your classifier predicts the right label for an obvious example.

`test_my_project.py` in the starter folder has examples. Run your tests from Thonny's **Tools** → **Open system shell…** with `python3 -m pytest -q`.

7. Playtest feedback card

Give this to your playtester (week 11). Then fix the most important thing first.

Reflection: Playtest card

Project: _____ Playtester (nickname): _____

Star 1 (something great):

Star 2 (something else great):

A wish (one thing to improve):

Did anything confuse you or break? Could someone who is color-blind play it?

8. How your project is judged

Your teacher uses the same rubric. Before the showcase, check one box in each row: where do *you* think you are?

	Just starting	Getting there	Got it	Expert
Creativity	☐ My idea is a copy of an example	☐ I changed an example a bit	☐ My project has my own idea and style	☐ My project surprises people with something new
Working code	☐ It crashes often	☐ It runs, with some bugs	☐ All must-have features work	☐ It works well, even when people do unexpected things
Design / OOP	☐ One class, or classes that do not help	☐ 2 classes, but jobs are mixed up	☐ Clear classes; inheritance or composition used well	☐ I can show why my class design makes the code easier to change
Testing	☐ Fewer than 3 tests	☐ 3–4 tests, or tests that fail	☐ 5+ passing tests of my logic	☐ My tests include edge cases and caught a real bug
Git history	☐ Fewer than 4 commits	☐ 4–7 commits, or unclear messages	☐ 8+ commits with clear messages	☐ My log tells the story of my project step by step
Explaining design decisions	☐ I can show what it does	☐ I can explain how one part works	☐ I can explain why I chose my design	☐ I can compare my design with another way and explain the trade-off

9. Need more time, or want more?

Differentiation

↓ Support path (needs more time): Start from a starter template and keep it small: 2 classes, 3 features, 5 simple tests (one per method). Ask your teacher for a "feature card" with one small step at a time. It is fine to finish fewer features well.
↑ Extension path (finished early): Add a second level or mode loaded from a file; write tests *before* the code (test-first); use BFS or recursion in a second place; add a settings dictionary saved as JSON; try k-nearest neighbors (3 vote) for option (c).

10. Showcase: your 3-minute talk

- ☐ **What** is it and who is it for? (20 seconds)
- ☐ **Demo**: show the best feature. (60 seconds)
- ☐ **A design decision**: what you chose, another way you could have done it, and why you chose yours. (45 seconds)
- ☐ **A bug story**: a bug you found and how you fixed it (maybe a test caught it!). (30 seconds)
- ☐ **Next**: what you would add next. (15 seconds)

Digital citizenship & safety: Showcase safety

Show your nickname, never your real name, on screen. Photos or videos only if everyone agrees. Your project stays on our computers; we do not upload it anywhere.